

AI LAB ASSISGNMENT 13.3

NAME:J.Anjali

ROLL NO:2403A510F0

BATCH:05

Task 1 – Remove Repetition

Prompt:

“Refactor a function that calculates the area of different shapes using repetitive if-elif statements. Improve the design by creating separate functions for each shape and using a dictionary-based dispatch to select the correct function, making the code cleaner and easier to maintain.”.

Code

```
from typing import List  Untitled-1  # Lab 13 – Code Refactoring: Improving L  Untitled-2

1  # Lab 13 – Code Refactoring: Improving Legacy Code with AI
2  # -----
3  # This file contains all 4 refactored tasks from Lab 13.
4  # Each task includes sample usage so you can run and test directly.
5
6  import math
7
8  # -----
9  # Task 1 – Remove Repetition
10 # -----
11 def area_rectangle(x, y):
12     return x * y
13
14 def area_square(x):
15     return x * x
16
17 def area_circle(r):
18     return math.pi * r * r
19
20 area_dispatch = {
21     "rectangle": lambda x, y: area_rectangle(x, y),
22     "square": lambda x, y=0: area_square(x),
23     "circle": lambda x, y=0: area_circle(x)
24 }
25
26 def calculate_area(shape, x, y=0):
27     """Calculate area of rectangle, square, or circle."""
28     if shape not in area_dispatch:
29         raise ValueError("Unsupported shape")
30     return area_dispatch[shape](x, y)
31
32 print("Task 1 Results:")
33 print("Rectangle (5x10):", calculate_area("rectangle", 5, 10))
34 print("Square (4x4):", calculate_area("square", 4))
35 print("Circle (r=3):", calculate_area("circle", 3))
36 print(f"-" * 40)
```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

-----
PS D:\AI LAB> python -u "d:\AI LAB\tempCodeRunnerFile.python"
Task 1 Results:
Rectangle (5x10): 50
Square (4x4): 16
Circle (r=3): 28.274333882308138
-----
PS D:\AI LAB>
```

Observation:

The refactored code eliminates repetition by using helper functions and a dictionary-based dispatch system. It is modular, easier to extend, and more readable.

Task 2 – Error Handling in Legacy Code

Prompt:

“Improve a file reading function that currently crashes when the file does not exist or there is an unexpected error. Use Python’s with statement and exception handling to make it safe and user-friendly.”

Code :

```
task3.py > ...
1  def read_file(filename):
2      try:
3          with open(filename, "r") as f:
4              data = f.read()
5              return data
6      except FileNotFoundError:
7          return "Error: File not found."
8      except Exception as e:
9          return f"Error: {e}"
10
11  # Example usage
12  if __name__ == "__main__":
13      # Create sample file for testing
14      with open("sample.txt", "w") as f:
15          f.write("Hello, this is a test file.\n")
16
17      # Call the function and print output
18      print(read_file("sample.txt"))      # Prints file contents
19      print(read_file("missing.txt"))    # Prints error message
20
```

Output Example:

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS D:\AI LAB> python -u "d:\AI LAB\task3.py"  
Hello, this is a test file.  
  
Error: File not found.  
PS D:\AI LAB>
```

Observation:

The refactored function ensures the program does not crash when reading files. Using `with open()` guarantees proper resource management, and `try-except` handles both missing files and unexpected errors, improving robustness and user experience.”

Task 3 – Complex Refactoring (Class)

Prompt:

“Refactor a student class that stores individual marks as separate variables. Improve code readability, maintainability, and scalability by using a list for marks, better variable names, and adding docstrings.”

Code :

```

task33.py > ...
1  # Task 3 - Complex Refactoring (Student Class)
2  # -----
3
4  class Student:
5      """
6      Represents a student with a name, age, and a list of marks.
7
8      Attributes:
9          name (str): The name of the student.
10         age (int): The age of the student.
11         marks (list): A list of integers representing the student's marks.
12     """
13
14     def __init__(self, name, age, marks):
15         """Initialize a Student object with name, age, and marks."""
16         self.name = name
17         self.age = age
18         self.marks = marks # store marks as a list
19
20     def details(self):
21         """Print the student's details."""
22         print(f"Name: {self.name}, Age: {self.age}")
23
24     def total(self):
25         """Return the total of all marks."""
26         return sum(self.marks)
27
28
29 # ----- Example Usage -----
30 if __name__ == "__main__":
31     # Create a student object
32     student1 = Student("Alice", 20, [85, 90, 95])
33
34     # Print student details
35     student1.details() # Output: Name: Alice, Age: 20
36
37     # Print total marks
38     print("Total Marks:", student1.total()) # Output: Total Marks: 270
39     # Create another student object

```

Output Example:

```

PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\AI LAB> python -u "d:\AI LAB\task33.py"
Name: Alice, Age: 20
Total Marks: 270
PS D:\AI LAB>

```

Observation:

The refactored class is more modular and readable. Storing marks in a list allows easy extension to more subjects. Method names and docstrings improve code understanding, and using `sum(self.marks)` simplifies total calculation."

Task 4 – Inefficient Loop Refactoring

Prompt:

"Optimize a loop that calculates the squares of numbers using a more Pythonic approach. Refactor it into a concise, readable format while maintaining the same output."

Code :

```
task34.py > square_numbers
1  # Task 4 – Inefficient Loop Refactoring
2  # -----
3
4  def square_numbers(nums):
5      """
6      Returns a list of squares of the numbers using list comprehension.
7
8      Args:
9      |   nums (list): A list of integers.
10     |
11     Returns:
12     |   list: Squares of the integers.
13     """
14     return [i * i for i in nums]
15
16
17  # ----- Example Usage -----
18  if __name__ == "__main__":
19      # Input list
20      nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
21
22      # Get squares
23      squares = square_numbers(nums)
24
25      # Print input and output
26      print("Input Numbers:", nums)
27      print("Squares:", squares)
```

Output Example:

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS D:\AI LAB> python -u "d:\AI LAB\task34.py"  
Input Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]  
PS D:\AI LAB>
```

Observation:

“Using a list comprehension reduces code length and improves readability. The functionality remains identical, but performance is slightly better and the code is more idiomatic to Python standards.”